

Orbit Wars — RL Formulation and GRPO Training Math

Native C++/LibTorch stack

June 4, 2026

This document specifies the policy, the reward, and the *Group Relative Policy Optimization* (GRPO) update used by the native trainer. Symbols map to `native/rl/config.hpp` and `native/rl/model/`.

Contents

1	Problem setup	1
2	Observation encoding	2
3	Policy network π_θ	3
4	Action distribution	5
5	Reward and return	5
6	Curriculum	6
7	GRPO update	7
7.1	Grouped sampling	7
7.2	Group-relative advantage (no critic)	7
7.3	Clipped policy surrogate	7
7.4	KL anchor to a reference policy	8
7.5	Total objective	8
7.6	Optimization	8
8	Self-play league	8
9	Symbols $\leftrightarrow$ config.hpp	9
A	Appendix: the PPO/GAE method being replaced	9

1 Problem setup

Orbit Wars is an N -player symmetric Markov game with $N \in \{2, 4\}$. The engine sets $N = |\text{state}|$ (`num_agents`) and assigns N home planets from a 4-fold-symmetric quadrant group ($N = 2$: two opposite quadrants; $N = 4$: all four). Each turn all players *simultaneously* issue a set of fleet *launches*; the deterministic engine then moves fleets, resolves collisions/combat, rotates planets,

produces ships, and (on scheduled steps) spawns comets. Scoring is a **free-for-all**: at termination player i 's score is its total ship count ships_i (planets + fleets), and the seat(s) with the maximum score receive reward +1, all others -1 (so $N = 4$ is winner-take-all among four individuals — not teams).

Training scope of this document. We currently train the **two-player special case** ($N = 2$: ego $p = 0$ against one opponent in the opposite quadrant). The native world generator places exactly two homes; the shaped reward (Section 5) uses the zero-sum margin $\text{ships}_0 - \text{ships}_1$; and each group plays a single opponent (scripted, or a frozen snapshot, Section 8). The policy (Section 3) and the GRPO update (Section 7) are *N-agnostic*; lifting to 4-player FFA requires only (i) 4-home world generation, (ii) replacing the zero-sum margin with the FFA return (max-score seat +1 else -1, or a rank-based shaping), and (iii) seating ≥ 1 extra opponents per game. This 2p→4p gap is an open modeling item, revisited in Section 5.

Factored action space. A turn's action is *factored over the ego player's planets* ("entities"). The observation exposes up to E entity slots ($E = 64$). Each owned planet e independently chooses a class

$$a_e \in \{0, 1, \dots, A - 1\}, \quad A = 1 + C |\mathcal{F}|, \quad (1)$$

where class 0 is *no-op*, $\mathcal{F} = \{0.25, 0.5, 0.75, 1.0\}$ is the set of ship-fraction choices, and C is the number of target choices: $C = E$ in *target mode* (each class ≥ 1 names a target entity t and a fraction f , with aim angle computed analytically to hit t), or $C = \text{angle_bins}$ in *angle mode* (class names an angle bin and a fraction). The class index for a launch is

$$\text{class}(t, f) = 1 + t |\mathcal{F}| + f. \quad (2)$$

2 Observation encoding

For ego player p and state s , the encoder (`native/core/encode.hpp`) emits

- entity features $X \in \mathbb{R}^{E \times F}$ ($F = 20$): per planet geometry, ownership (ego/ally/enemy/neutral one-hots), ships, production, comet/orbit flags, proximity fleet pressure, and **five incoming-fleet threat features** (below);
- entity mask $m \in \{0, 1\}^E$: $m_e = 1$ iff slot e is a real entity;
- action mask $u \in \{0, 1\}^E$: $u_e = 1$ iff e is an ego-owned planet with > 0 ships (i.e. legally able to launch);
- globals $g \in \mathbb{R}^G$ ($G = 10$): step/phase, board-wide ship and production margins, counts, angular velocity, etc.

Incoming-fleet threat features (defensive lookahead). A purely *reactive* policy cannot defend if it cannot see what is coming: the base encoder summarized fleets only as a radius-25 proximity sum, which is blind to a fleet's *heading* (one aimed away is harmless; one aimed in from afar is the danger). We add a closed-form projection of each in-flight fleet onto the planets. A fleet at position o with unit heading $\hat{d} = (\cos \alpha, \sin \alpha)$ and ship count ν moves at speed

$$v(\nu) = \min\left(v_{\max}, 1 + (v_{\max} - 1)\left(\frac{\ln \max(1, \nu)}{\ln 1000}\right)^{1.5}\right), \quad v_{\max} = 6, \quad (3)$$

matching the engine’s fleet dynamics. For a planet at c with radius ϱ , the ray $o + t\hat{d}$ enters the planet disk at

$$t_{\text{ca}} = -\langle o - c, \hat{d} \rangle, \quad p^2 = \|o - c\|^2 - t_{\text{ca}}^2, \quad t_{\text{hit}} = t_{\text{ca}} - \sqrt{\varrho^2 - p^2} \quad (\text{if } t_{\text{ca}} \geq 0, p^2 \leq \varrho^2). \quad (4)$$

The fleet is assigned to the planet with the smallest $t_{\text{hit}} \geq 0$, unless the ray first crosses the sun (same test against the central disk), in which case it is dropped — the heuristic “does this fleet hit anything?” filter. Its ETA in ticks is $\tau = t_{\text{hit}}/v(\nu)$. Aggregating per planet e over enemy ($\mathcal{E}_e$) and ally ($\mathcal{A}_e$) inbound fleets, with $I_e^{\text{en}} = \sum_{\mathcal{E}_e} \nu$, $I_e^{\text{al}} = \sum_{\mathcal{A}_e} \nu$, and $\tau_e^{\text{en}}, \tau_e^{\text{al}}$ the minimum ETAs, the five features are

$$\left[\widetilde{I}_e^{\text{en}}, e^{-\tau_e^{\text{en}}/\theta}, \widetilde{I}_e^{\text{al}}, e^{-\tau_e^{\text{al}}/\theta}, \tanh(\Delta_e/\theta_m) \right], \quad (5)$$

where $\widetilde{x} = \ln(1+x)/\ln 1000$ is the ship-log compressor, $\theta = 8$ sets the imminence half-life, and $\Delta_e = \text{ships}_e + \text{prod}_e \tau_e^{\text{en}} + I_e^{\text{al}} - I_e^{\text{en}}$ is the *hold-margin*: ships on hand, plus production banked until impact, plus reinforcements, minus the attack (> 0 holds, < 0 falls). Imminence and margin are zeroed when there is no enemy threat. The projection is an approximation (it ignores planet rotation), but it is computed *identically* in C++ and Python (parity-tested bit-for-bit), so the policy trained natively behaves identically when served from pure Python.

3 Policy network π_θ

The network (`native/rl/model/policy_net.hpp`) maps (X, m, u, g) to per-entity class logits $Z \in \mathbb{R}^{E \times A}$. It has **no value head**; GRPO needs no critic. Let $d = 128$ (**hidden**). The trunk is *MLP* + *residual GLU* (gated linear unit): each encoder is a two-layer ReLU MLP followed by one residual GLU block, which adds multiplicative gating and a skip connection on top of the MLP features.

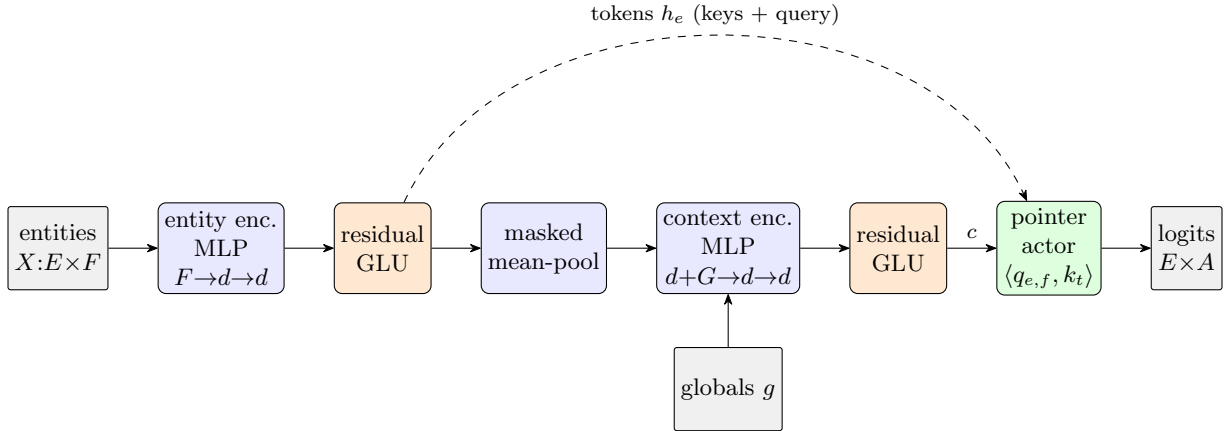


Figure 1: Policy network data flow (target mode). Each encoder is a two-layer MLP followed by a residual GLU block (Fig. 2). The masked mean-pool of the entity tokens h_e , concatenated with the globals g , gives the board context c ; the pointer actor scores every (launcher e , target t , fraction f) from h_e and c (plus a per-launcher no-op), producing the $E \times A$ logits, which are then legality-masked.

Residual GLU block. For an input $z \in \mathbb{R}^d$, with three $d \times d$ weight matrices (gate W_g , value W_v , output W_o),

$$\text{GLU}(z) = (W_v z + b_v) \odot \sigma(W_g z + b_g), \quad \mathcal{B}(z) = z + W_o \text{GLU}(z) + b_o \in \mathbb{R}^d, \quad (6)$$

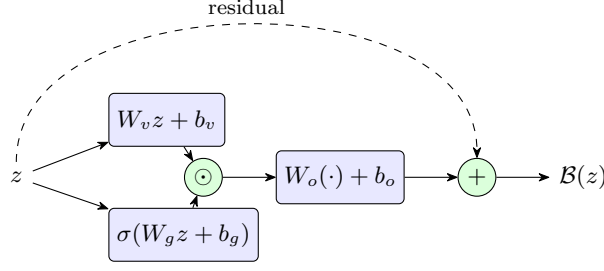


Figure 2: The residual GLU block $\mathcal{B}$ (Eq. 6): a value branch $W_v z$ and a sigmoid gate $\sigma(W_g z)$ are multiplied elementwise ($\odot$), projected by W_o , and added back to the input (the residual skip). One such block (its own $d \times d$ weights) follows each encoder MLP.

where σ is the logistic sigmoid and $\odot$ is elementwise. The sigmoid gate lets the block pass, attenuate, or suppress each feature channel conditionally (e.g. amplify the incoming-threat channels of Section 2 when a planet is under attack); the residual keeps optimization easy. Applied per entity row (encoder 1) or to the pooled context vector (encoder 2), it shares weights across rows just like the MLP.

Entity encoder (2-layer MLP + GLU). A shared two-layer MLP over each entity row, then a residual GLU block ($\mathcal{B}_{\text{ent}}$, its own weights):

$$\tilde{h}_e = W_2 \text{ReLU}(W_1 x_e + b_1) + b_2, \quad h_e = \mathcal{B}_{\text{ent}}(\tilde{h}_e) \in \mathbb{R}^d, \quad e = 1, \dots, E. \quad (7)$$

Context encoder (2-layer MLP + GLU). A mask-aware mean pool, a two-layer MLP on the pooled tokens concatenated with the globals, then its own residual GLU block $\mathcal{B}_{\text{ctx}}$:

$$\bar{h} = \frac{\sum_e m_e h_e}{\max(\sum_e m_e, 1)}, \quad \tilde{c} = W_4 \text{ReLU}(W_3 [\bar{h}; g] + b_3) + b_4, \quad c = \mathcal{B}_{\text{ctx}}(\tilde{c}) \in \mathbb{R}^d. \quad (8)$$

Actor head. Two interchangeable heads produce the raw logits $\tilde{Z}$:

Angle mode: an MLP on each entity token concatenated with the context,

$$\tilde{z}_e = W_6 \text{ReLU}(W_5 [h_e; c] + b_5) + b_6 \in \mathbb{R}^A. \quad (9)$$

Target mode (pointer actor): for each launcher e and fraction f a query $q_{e,f} \in \mathbb{R}^d$ (from $W_q [h_e; c]$ reshaped to $|\mathcal{F}| \times d$), for each target t a key $k_t = W_k h_t$, and a per-launcher no-op logit $n_e = w_n^\top [h_e; c]$:

$$\text{score}(e, t, f) = \frac{\langle q_{e,f}, k_t \rangle}{\sqrt{d}}, \quad \tilde{z}_{e,0} = n_e, \quad \tilde{z}_{e,\text{class}(t,f)} = \text{score}(e, t, f). \quad (10)$$

The score depends on *both* launcher and target encodings, so the head can rank targets by geometry (the property a plain MLP actor lacks).

Legality masking. Illegal classes are set to $-\infty$ (numerically -10^9). Class 0 (no-op) is always legal. For a launch class (t, f) ,

$$Z_{e,\text{class}(t,f)} = \begin{cases} \tilde{z}_{e,\text{class}(t,f)} & \text{if } u_e = 1 \wedge m_t = 1 \wedge t \neq e \quad (\text{target mode}), \\ \tilde{z}_{e,\text{class}(t,f)} & \text{if } u_e = 1 \quad (\text{angle mode}), \\ -\infty & \text{otherwise.} \end{cases} \quad (11)$$

Layer inventory (target mode, $d = 128$). All layers are `nn.Linear`; the trunk is the two MLPs each followed by one residual GLU block, then the pointer actor.

Module	Layers (name: shape)	Count
Entity encoder	<code>ent0: $F \rightarrow d$, ent2: $d \rightarrow d$; GLU ent_glu_{gate, val, out}: $d \rightarrow d$</code>	5
Context encoder	<code>ctx0: $d + G \rightarrow d$, ctx2: $d \rightarrow d$; GLU ctx_glu_{gate, val, out}: $d \rightarrow d$</code>	5
Pointer actor	<code>aq: $2d \rightarrow \mathcal{F} d$, ak: $d \rightarrow d$, anoop: $2d \rightarrow 1$</code>	3
<i>Trainable total (no critic — GRPO)</i>		13

(The Python serving net additionally carries a 2-layer critic `critic.0/.2` for legacy PPO tooling; the native GRPO trainer and arena never instantiate or load it.) With $F=20$, $G=10$, $d=128$, $|\mathcal{F}|=4$ this is $\approx 0.18\text{M}$ parameters.

4 Action distribution

Each entity is an independent masked categorical; a turn’s joint action is their product over *real* entities (`native/rl/model/distribution.hpp`):

$$\pi_\theta(a_e \mid s) = \text{softmax}(Z_e)_{a_e}, \quad \pi_\theta(a \mid s) = \prod_{e: m_e=1} \pi_\theta(a_e \mid s). \quad (12)$$

Hence the joint log-probability and entropy are entity sums:

$$\log \pi_\theta(a \mid s) = \sum_e m_e \log \text{softmax}(Z_e)_{a_e}, \quad (13)$$

$$\mathcal{H}[\pi_\theta(\cdot \mid s)] = \sum_e m_e \left(- \sum_{a'} \text{softmax}(Z_e)_{a'} \log \text{softmax}(Z_e)_{a'} \right). \quad (14)$$

Sampling draws $a_e \sim \text{Categorical}(\text{softmax}(Z_e))$ per entity (greedy uses $\arg \max_a Z_{e,a}$ at evaluation).

5 Reward and return

Production-only dense reward. The shaped reward rewards *growing one’s own production efficiently*, and nothing else — not raw planet count, not ship count (both of which an agent can maximize while still losing the game). Let $\Pi_0(s) = \sum_{p: \text{owner}(p)=0} \text{prod}(p)$ be ego’s total planetary production, read directly from the state. For the ego launches $\mathcal{L}_t$ issued at step t , classify each by the closed-form projection of Section 2: it *hits* if its fleet reaches some planet, else it is *wasted*. With h_t, m_t the hit/wasted counts, weights w_Π (`prod_reward_weight`), ρ^+ (`launch_hit_reward`), ρ^- (`launch_miss_penalty`), and scale/clip σ, κ ,

$$r_t = \text{clip} \left(\frac{w_\Pi(\Pi_0(s_{t+1}) - \Pi_0(s_t)) + \rho^+ h_t - \rho^- m_t}{\sigma}, -\kappa, \kappa \right). \quad (15)$$

Capturing a high-production planet is rewarded *exactly* through $\Delta \Pi_0$; a fleet launched to land nowhere is penalized the instant it is fired; a productive launch earns a small bonus (which also pulls the policy off the passive “never launch” optimum).

Outcome and the loss forfeit. At termination the game is scored by total ships; $o_i = +1$ win, -1 loss, 0 draw. Let $D_i = \sum_t \gamma^t r_{i,t}$ be the accumulated discounted dense reward. It is kept on a win but **forfeited on a loss** — the return negates it, so a trajectory that hoarded production yet still lost scores a large *negative* (`loss_forfeit`; $w_d = \text{dense_weight}$, $w_o = \text{outcome_weight}$):

$$R_i = \begin{cases} w_d D_i + w_o, & o_i > 0 \text{ (win),} \\ w_d D_i, & o_i = 0 \text{ (draw),} \\ -w_d D_i - w_o, & o_i < 0 \text{ (loss).} \end{cases} \quad (16)$$

Winning thus dominates by at least $2w_o$, and among losses the *more* a trajectory accumulated without converting it to a win, the worse it scores — directly penalizing “maximize my own production/ships but lose the game,” which earlier shaped rewards failed to discourage. Setting `loss_forfeit=false` recovers the plain $R_i = w_d D_i + w_o o_i$.

Note — 2p vs. 4p objective. The margin potential Φ and the outcome o_i are the two-player ($N = 2$) zero-sum forms. In $N = 4$ free-for-all the objective is *ordinal* — be the maximum of four — which is not zero-sum: e.g. Φ would become $\text{ships}_0 - \max_{k \neq 0} \text{ships}_k$ (margin over the *strongest* rival, or a soft-max thereof) and o_i the FFA win/lose of Section 1. The GRPO machinery (advantage, surrogate, KL) is unchanged — only R_i in Eq. (16) is redefined.

6 Curriculum

From-scratch and BC-finetuned RL repeatedly plateaued at $\sim$ random level *versus* the scripted **starter** ($\leq 2\%$ win), because two skills are entangled in the full adversarial task: (i) *aim + expand* — launch ships that actually capture planets — and (ii) *defend* — hold planets against incoming fleets. We decouple them with a three-stage curriculum (`RolloutConfig::stage`); each stage warm-starts from the previous stage’s policy, which also becomes the new KL reference π_{ref} of Section 7.4 (anchor-and-advance, rather than anchoring forever to the passive BC). *All stages share the one reward of Section 5* (production-only dense + launch quality, loss-forfeit return); only the **opponent** and the discount γ change across stages.

Stage 1 — solo expansion (passive opponent). The opponent issues no launches, so the policy can freely learn to aim and to take the whole map without adversarial punishment. Because the passive opponent hoards production on its home planet, the ego must actually *conquer* it (not merely grab neutrals) to win the terminal ship score — failing that, the loss forfeit (Eq. (16)) turns the episode negative. Running with $\gamma < 1$ ($\gamma = 0.99$) makes *earlier* production gains worth more, i.e. a discounted “planet-takeover-time” incentive for fastest full-map conquest; the episode ends early once the ego owns everything.

Stage 2 — 1v1 vs. starter. The opponent is always **starter** and $\gamma = 1$ (timing is now dictated by the opponent, not a solo clock). The same reward applies, so the loss forfeit is fully in force: the agent must convert its production into an actual win and must *defend* (using the threat features of Section 2) or forfeit everything it built. The Stage-1 policy supplies both the initialization and π_{ref} , so it keeps its aiming/expansion skill while learning to win.

Stage 3 — mixed opponents. The opponent per group is sampled from `{random, starter, (league snapshots)}` as in Section 8, generalizing beyond a single fixed adversary toward the multi-opponent FFA setting.

7 GRPO update

GRPO replaces the value baseline of actor–critic methods with a *group-relative* baseline: the policy plays a group of trajectories from the *same* start against the *same* opponent, and each trajectory’s return is centered against its group.

7.1 Grouped sampling

For iteration we draw N groups (`num_groups`); group j fixes an initial state $s_0^{(j)}$ (a world) and an opponent, and rolls out G trajectories (`group_size`) under the current behavior policy $\pi_{\theta_{\text{old}}}$ (stochastic). The only within-group variation is the policy’s own action sampling, so the group mean is a valid, low-variance baseline. The env count is $N \cdot G$ (`num_envs`). For every transition we store (X, m, u, g) , the action $a_{i,t}$, the behavior log-prob $\log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})$, and the reference log-prob $\log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t})$ (Section 7.4).

7.2 Group-relative advantage (no critic)

Let $\mathcal{G}_j$ be the trajectories of group j , with mean and standard deviation

$$\mu_j = \frac{1}{G} \sum_{i \in \mathcal{G}_j} R_i, \quad \varsigma_j = \sqrt{\frac{1}{G} \sum_{i \in \mathcal{G}_j} (R_i - \mu_j)^2}. \quad (17)$$

The advantage of trajectory i (group $j = j(i)$), *broadcast to all of its timesteps*, is

$$\hat{A}_i = \begin{cases} \frac{R_i - \mu_j}{\varsigma_j + \varepsilon}, & \text{whiten_advantage} = \text{true}, \\ R_i - \mu_j, & \text{otherwise,} \end{cases} \quad (18)$$

with floor $\varepsilon = \text{adv_eps}$. There is no learned $V(s)$ and no temporal-difference or GAE term — the entire baseline is Eq. (18).

7.3 Clipped policy surrogate

With the per-step importance ratio

$$\rho_{i,t}(\theta) = \exp\left(\log \pi_{\theta}(a_{i,t} \mid s_{i,t}) - \log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})\right), \quad (19)$$

(each log-prob the entity sum of Eq. (13)), the surrogate is the PPO clip objective

$$\mathcal{L}^{\text{clip}}(\theta) = - \frac{1}{\sum_i T_i} \sum_{i,t} \min\left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right), \quad (20)$$

with clip range $\epsilon = \text{clip}$.

7.4 KL anchor to a reference policy

A reference policy π_{ref} (the fixed behavior-cloning checkpoint) anchors training, which counteracts the empirically observed drift of plain PPO away from the strong BC behavior. We penalize the unbiased, non-negative k_3 estimator of $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$. With $\Delta_{i,t} = \log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t}) - \log \pi_\theta(a_{i,t} \mid s_{i,t})$,

$$\widehat{\text{KL}}_{i,t} = \exp(\Delta_{i,t}) - \Delta_{i,t} - 1 \quad (\geq 0), \quad \mathcal{L}^{\text{KL}}(\theta) = \frac{1}{\sum_i T_i} \sum_{i,t} \widehat{\text{KL}}_{i,t}. \quad (21)$$

7.5 Total objective

Combining the surrogate, the KL anchor (weight $\beta = \text{kl_beta}$) and an entropy bonus (weight $c_{\mathcal{H}} = \text{ent_coef}$, from Eq. (14)):

$$\boxed{\mathcal{L}(\theta) = \mathcal{L}^{\text{clip}}(\theta) + \beta \mathcal{L}^{\text{KL}}(\theta) - c_{\mathcal{H}} \overline{\mathcal{H}}[\pi_\theta]} \quad (22)$$

where $\overline{\mathcal{H}}$ is the mean entropy over transitions.

7.6 Optimization

For each iteration: collect the grouped trajectories under $\pi_{\theta_{\text{old}}} = \pi_\theta$; compute returns (16) and advantages (18); then for `update_epochs` passes over `minibatches` shuffled minibatches, take Adam steps on (22) with learning rate $\eta = \text{lr}$, $\epsilon_{\text{Adam}} = \text{adam_eps}$, and global gradient-norm clip `max_grad_norm`. Advantages are computed once per iteration (off $\pi_{\theta_{\text{old}}}$); the ratio (20) and KL (21) are recomputed each minibatch under the live π_θ .

8 Self-play league

Once training has reached `start_step`, the opponent for each group is sampled from a mix of `{random, starter, league}` with weights `(mix_random, mix_starter, mix_pool)`. The league is a bounded set (`pool_capacity`) of *historical* frozen snapshots — not a single latest self — which avoids the cycling/forgetting of single-snapshot self-play. Promotion is *win-gated*: a new snapshot is admitted only when the current policy beats the pool with win rate $\geq \text{promote_winrate}$, so the policy never trains against a regression of itself.

9 Symbols $\leftrightarrow$ config.hpp

Symbol	config.hpp field	Default
d	ModelConfig::hidden	128
E	ModelConfig::max_entities	64
$\mathcal{F}$	ModelConfig::fractions	{.25, .5, .75, 1}
G, N	GrpoConfig::group_size, num_groups	8, 64
ϵ	GrpoConfig::clip	0.2
β	GrpoConfig::kl_beta	0.04
$c_{\mathcal{H}}$	GrpoConfig::ent_coef	0.01
ε	GrpoConfig::adv_eps	10^{-4}
w_d, w_o	GrpoConfig::dense_weight, outcome_weight	1, 1
γ	GrpoConfig::gamma	1.0
w_{Π}	RolloutConfig::prod_reward_weight	10
ρ^+	RolloutConfig::launch_hit_reward	0.5
ρ^-	RolloutConfig::launch_miss_penalty	1.0
loss_forfeit	RolloutConfig::loss_forfeit	true
σ, κ	RolloutConfig::reward_scale, reward_clip	50, 5
stage	RolloutConfig::stage	2
θ, θ_m	threat τ -scale, margin-scale (encode.hpp)	8, 100
η	OptimConfig::lr	3×10^{-4}

A Appendix: the PPO/GAE method being replaced

The previous trainer (`native/rl/trainer.hpp`) used actor-critic PPO with a value head V_{ϕ} and Generalized Advantage Estimation. For completeness, with TD residual $\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$,

$$\hat{A}_t^{\text{GAE}} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad \mathcal{L}^V = \frac{1}{2} (V_{\phi}(s_t) - \hat{R}_t)^2, \quad \hat{R}_t = \hat{A}_t^{\text{GAE}} + V_{\phi}(s_t), \quad (23)$$

and total loss $\mathcal{L}^{\text{clip}} + c_V \mathcal{L}^V - c_{\mathcal{H}} \overline{\mathcal{H}}$. GRPO removes V_{ϕ} (and thus λ , c_V , and value warmup) entirely, replacing $\hat{A}_t^{\text{GAE}}$ with the group-relative $\hat{A}_i$ of Eq. (18).